NATS Server — Raft Implementation

An end-to-end technical walkthrough of the consensus layer that powers JetStream clustering: leader election & timers, heartbeats, the NATS subscription/wire protocol, and the full lifecycle of **publishing**, **consuming** and **acknowledging** a WorkQueue message — with a risk assessment against the Raft paper and field implementations.

`server/raft.go · 5,325 LOC` `server/jetstream_cluster.go` `server/consumer.go` `server/stream.go` `Generated 2026-06-09`

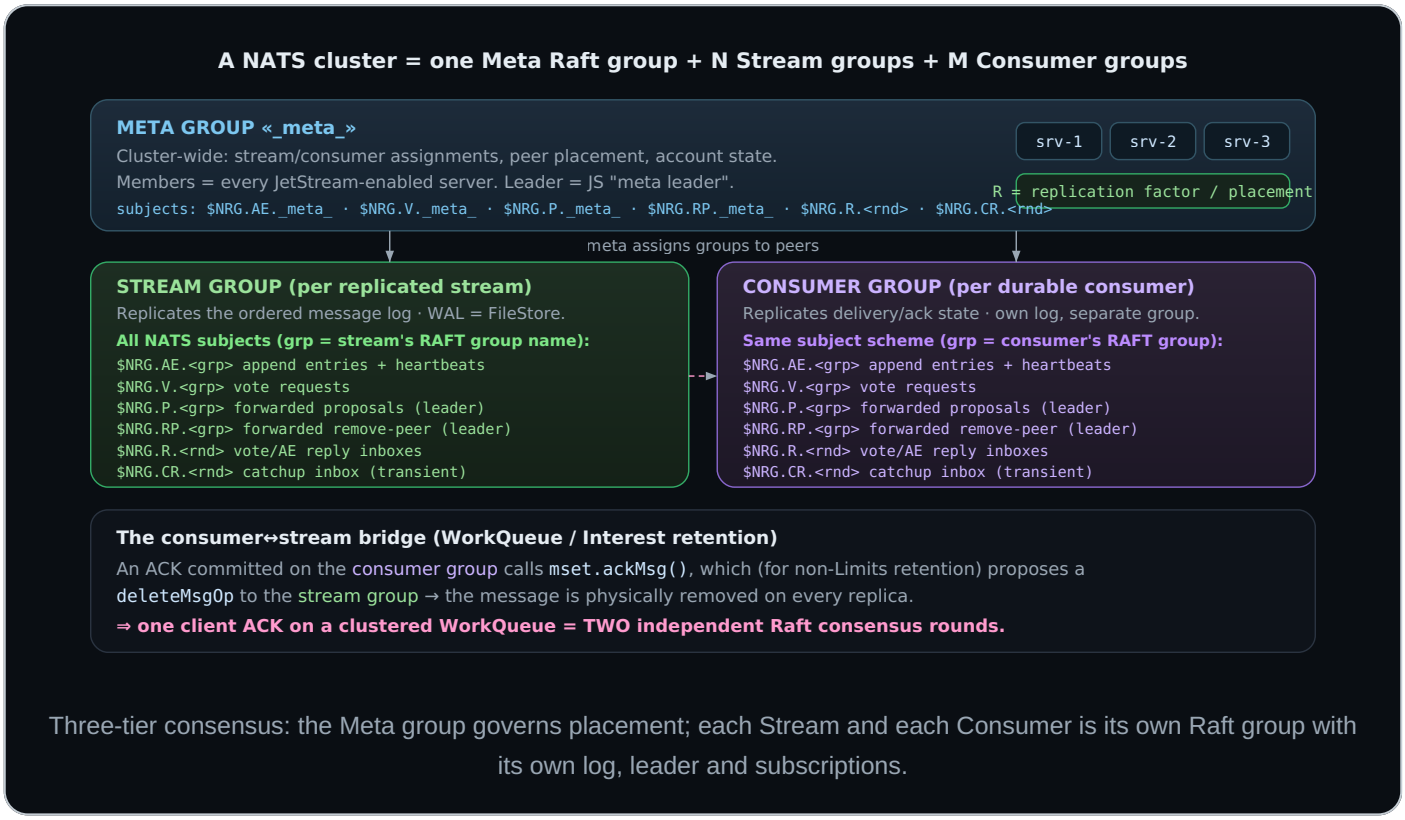
CONTENTS

END-TO-END FLOWS

OPERATIONAL

1 · Overview & topology

NATS implements **Raft from scratch** in `server/raft.go` rather than vendoring an external library. The novel twist: the transport is **NATS itself** — peers talk over subject-based pub/sub on an internal system client, and each Raft node's write-ahead log is a **JetStream file/mem store** (the same engine that stores stream messages). Consensus is used at three independent layers.



Three-tier consensus: the Meta group governs placement; each Stream and each Consumer is its own Raft group with its own log, leader and subscriptions.

3

independent Raft layers (meta / stream / consumer)

4–6

NATS subscriptions per Raft group (follower→leader)

4–9 s

randomized election timeout; 1 s heartbeat

2 · Raft groups, identity & storage

A Raft group is created by `js.createRaftGroup()` `jetstream_cluster.go:2794`. The node is bootstrapped (`bootstrapRaftNode`) and started (`startRaftNode`), then attached to its asset — `rg.node` for streams, `o.node = ca.Group.node` for consumers `consumer.go:1524`. A nil `node` is the universal "am I single-replica (R1)?" test.

Identity

- Node / peer IDs are **8 bytes** (`idLen = 8`), a truncated HighwayHash of the server name.
- Group name (`n.group`) keys all shared subjects.
- Term & vote persisted to `tav.idx`; peer set to `peers.idx` `raft.go:4856,4938`.

Write-ahead log (WAL)

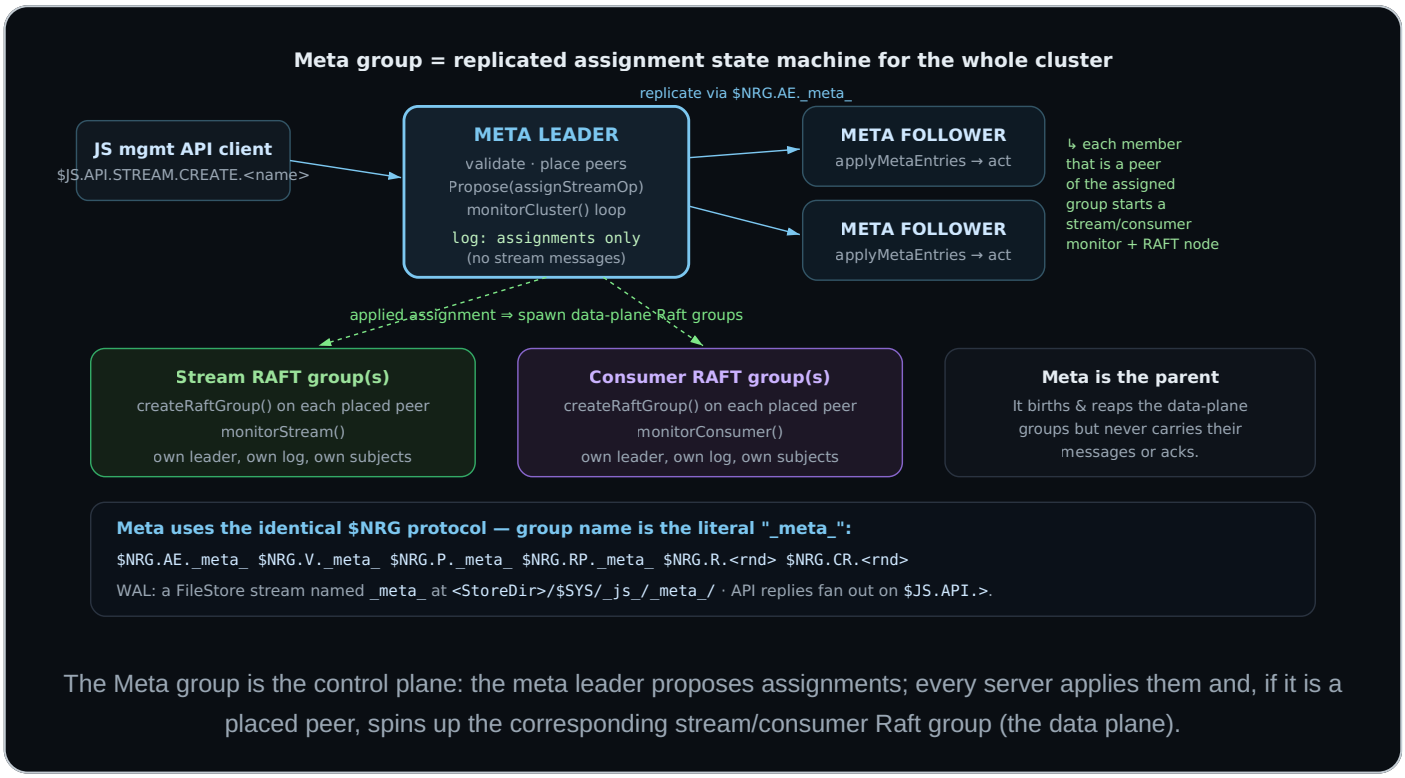
- The Raft log **is a JetStream store**: a `FileStore` (`defaultMediumBlockSize`) or `MemStore` `jetstream_cluster.go:2911-2929`.
- Each append entry is one "message" in that store, keyed by Raft index.
- Stored under `<StoreDir>/$SYS/_js_/<group>/`.

> All Raft traffic runs on a dedicated **internal system client** (`n.c`), normally on the **system account**; with account-NRG enabled and all peers capable, traffic moves to the asset's configured NRG account `raft.go:680-744`. Outbound messages go through a per-account send queue (`n.sq`, `sendq.go`) that injects them as if published by an internal client.

3 · The Meta group — the cluster's control plane

The **Meta group** (raft group name `_meta_`, `defaultMetaGroupName` `jetstream_cluster.go:457`) is a single, special Raft group whose **members are every JetStream-enabled server in the cluster**. It is not a data plane — it stores *no* stream messages. Instead its replicated log is the cluster's

authoritative map of "what runs where": which streams and consumers exist, their configs, and which peers host each stream/consumer Raft group. Its leader is the **"meta leader"** (`s.isMetaLeader`), the node that drives all placement decisions and answers the JetStream management API.



The Meta group is the control plane: the meta leader proposes assignments; every server applies them and, if it is a placed peer, spins up the corresponding stream/consumer Raft group (the data plane).

Responsibilities

What the meta log carries (meta-only entry ops)

Op	Meaning
`assignStreamOp`	create/place a stream on a peer set
`removeStreamOp`	delete a stream
`assignConsumerOp` / `assignCompressedConsumerOp`	create/place a consumer
`removeConsumerOp`	delete a consumer
`updateStreamOp`	reconfigure an existing stream
peer entries (`EntryAddPeer` / `EntryRemovePeer`)	cluster membership changes

Decoded & acted on in `applyMetaEntries`; entry ops at `jetstream_cluster.go:122-153`.

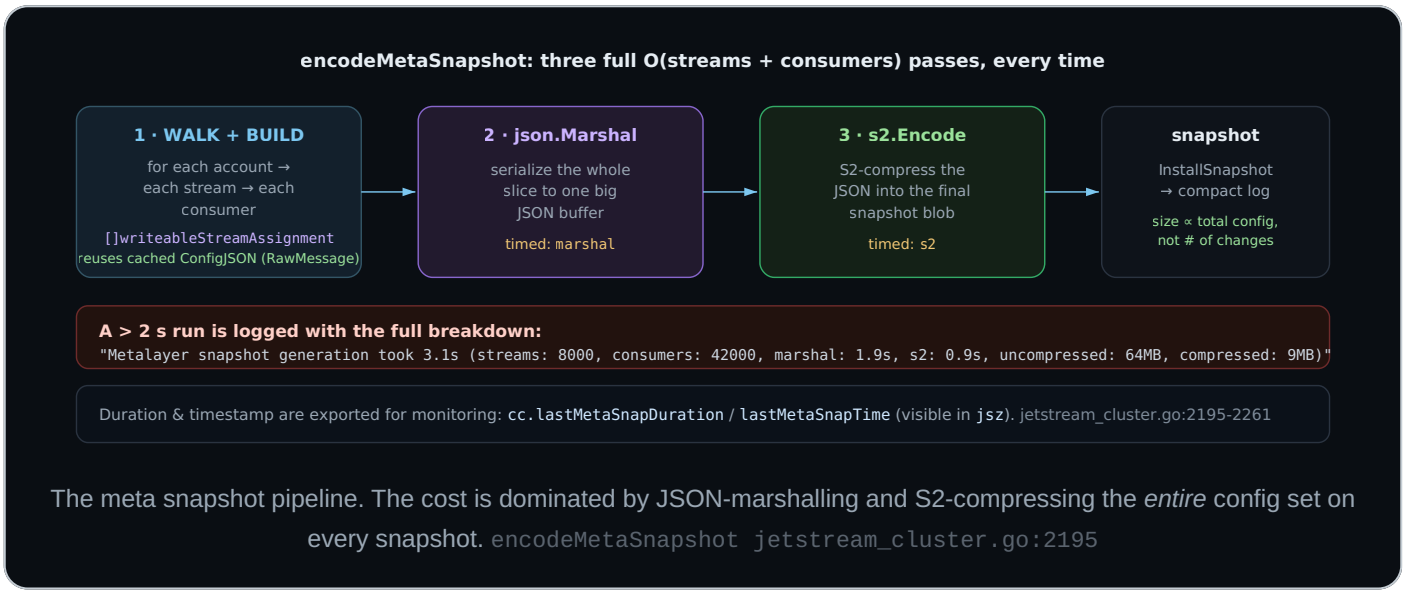
What the meta leader does

- **API gateway:** validates and serves `$JS.API.>` stream/consumer create/update/delete requests; only the leader proposes.
- **Placement:** chooses peer sets (RF, tags, balance) and writes them as assignments.
- **Reconciliation:** on apply, every server starts/stops the stream/consumer Raft groups it is a peer of (`processStreamAssignment` / `processConsumerAssignment`).
- **Membership & health:** tracks peers, runs `checkClusterSize()`, handles peer add/remove and mixed-mode boot sizing.
- **Orphan cleanup:** `checkForOrphans` 30 s after recovery.

> **Failure-domain separation.** Because the meta group is separate from every data group, losing meta *quorum* freezes the **control plane** (no new streams/consumers, no placement changes, no API mutations) but established streams keep serving reads/writes via their own independent Raft groups — and vice-versa. The meta monitor loop is `monitorCluster()` `jetstream_cluster.go:1574`.

The `metaSnapshot` operation — the cluster's heaviest serialization

Unlike a stream snapshot (which captures opaque message-store state), the meta snapshot is a **full serialization of the entire cluster's configuration**. There is no delta encoding: every snapshot re-serializes **every stream and every consumer in every account**, regardless of how little changed. On large clusters (tens of thousands of assets) this is the single most expensive recurring operation in the control plane.



The meta snapshot pipeline. The cost is dominated by JSON-marshalling and S2-compressing the *entire* config set on every snapshot. `encodeMetaSnapshot jetstream_cluster.go:2195`

Blocking vs. asynchronous — and the lock that matters

The two strategies differ in **one critical way: whether they hold the global JetStream lock (`js.mu`) while serializing**.

Blocking (`metaSnapshot()`)

- Takes `js.mu.RLock()` and calls `encodeMetaSnapshot(cc.streams)` on the **live** cluster map `jetstream_cluster.go:2010-2015`.
- The marshal + S2 compress run **while holding the JS read lock** — so for the whole duration, anything needing the write lock (new assignments, API mutations, placement) stalls.
- Used after repeated async failures, on shutdown, and when the log has grown past 10× the size threshold.

Async (default)

- Runs in its own goroutine and **never takes** `js.mu` (only brief Raft-node locks).
- Instead of reading live state, it reconstructs current state from **last snapshot + replayed log tail**: `LoadLastSnapshot` → `decodeMetaSnapshot` (JSON+S2 decode) → `collectStreamAndConsumerChanges` (replay entries via `AppendEntriesSeq`) → `encodeMetaSnapshot` `jetstream_cluster.go:1764-1796`.
- Pays an extra decode-then-replay cost to stay off the global lock — a deliberate latency-for-availability trade.

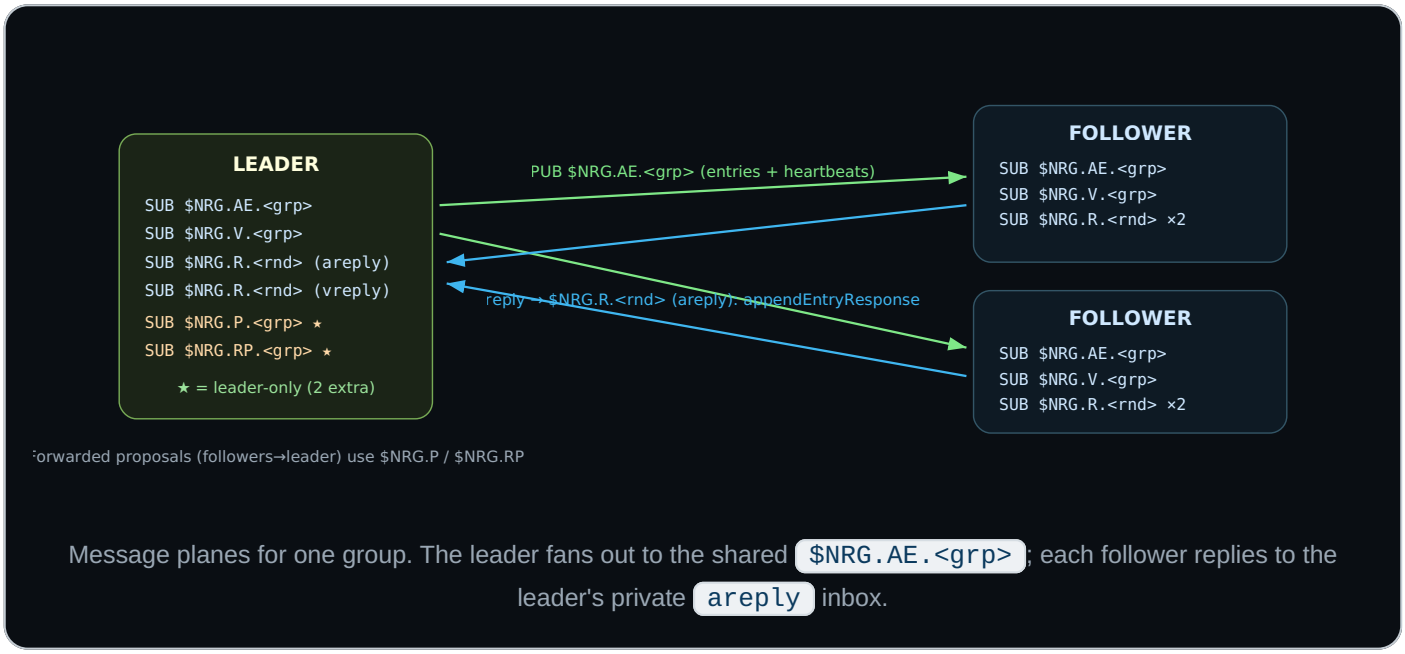
Why this is the expensive one. Because it is a *full* snapshot, cost grows with **total cluster size**, not with churn — adding one consumer to a 50k-consumer cluster still re-marshals and re-compresses all 50k. Two mitigations are already in place: per-asset configs are cached as pre-marshalled `json.RawMessage` (`ConfigJSON`) so structs aren't re-encoded, and the async path keeps it off `js.mu`. The residual risks: (1) the meta log compacts only as fast as snapshots complete, so a slow snapshot on a huge cluster lets the `_meta_` WAL grow; (2) if it degrades to the blocking path, a multi-second serialization stalls all control-plane operations cluster-wide. Watch `lastMetaSnapDuration` and the "snapshot generation took" warnings.

Meta-group recovery

On startup each server replays its `_meta_` WAL into a `recoveryUpdates` staging set; when the apply queue emits the catchup-complete nil guard, staged stream/consumer adds, updates and removals are applied in dependency order, `metaRecovering` is cleared, and a **forced compacting snapshot** is taken so the node starts with a small log `jetstream_cluster.go:1623-1786`.

4 · NATS subscriptions & subjects

Everything Raft does is one of a handful of subjects under the `$NRG` ("NATS Raft Group") prefix. Subjects keyed by `%s = group` are **shared** (all peers subscribe); `reply` subjects are **private inboxes** with an 8-char random base-62 suffix. **All are plain subscriptions — never queue groups** `raft.go:2331-2336`.



Message planes for one group. The leader fans out to the shared `$NRG.AE.<grp>`; each follower replies to the leader's private `areply` inbox.

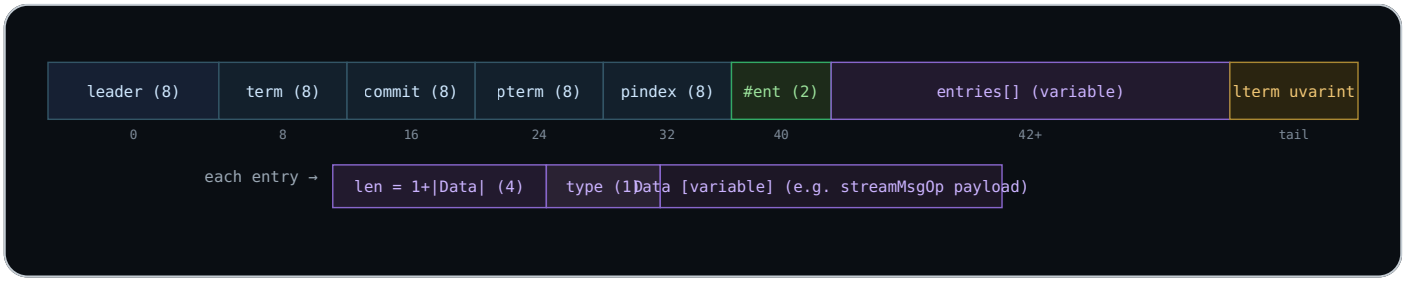
Field	Subject pattern	Handler	Carries	Who subs
`asubj`	`$NRG.AE.<grp>`	`handleAppendEntry`	append entries + heartbeats (from leader)	all
`areply`	`$NRG.R.<rnd>`	`handleAppendEntryResponse`	per-entry success/fail acks	own inbox
`vsubj`	`$NRG.V.<grp>`	`handleVoteRequest`	RequestVote RPCs	all
`vreply`	`$NRG.R.<rnd>`	`handleVoteResponse`	vote grants/denials	own inbox
`psubj`	`$NRG.P.<grp>`	`handleForwardedProposal`	proposals forwarded by followers	`leader only`
`rpsubj`	`$NRG.RP.<grp>`	`handleForwardedRemovePeerProposal`	remove-peer proposals	`leader only`
catchup	`$NRG.CR.<rnd>`	`handleAppendEntry`	streamed log catchup + snapshots	`transient`

Inbound messages are decoded and pushed onto in-process queues (`ipQueue`): `entry`, `resp`, `reqs`, `votes`, `prop` — and drained by the single state-machine goroutine (`runAsFollower` / `runAsCandidate` / `runAsLeader`). Committed entries leave via the `apply` queue (`ApplyQ()`).

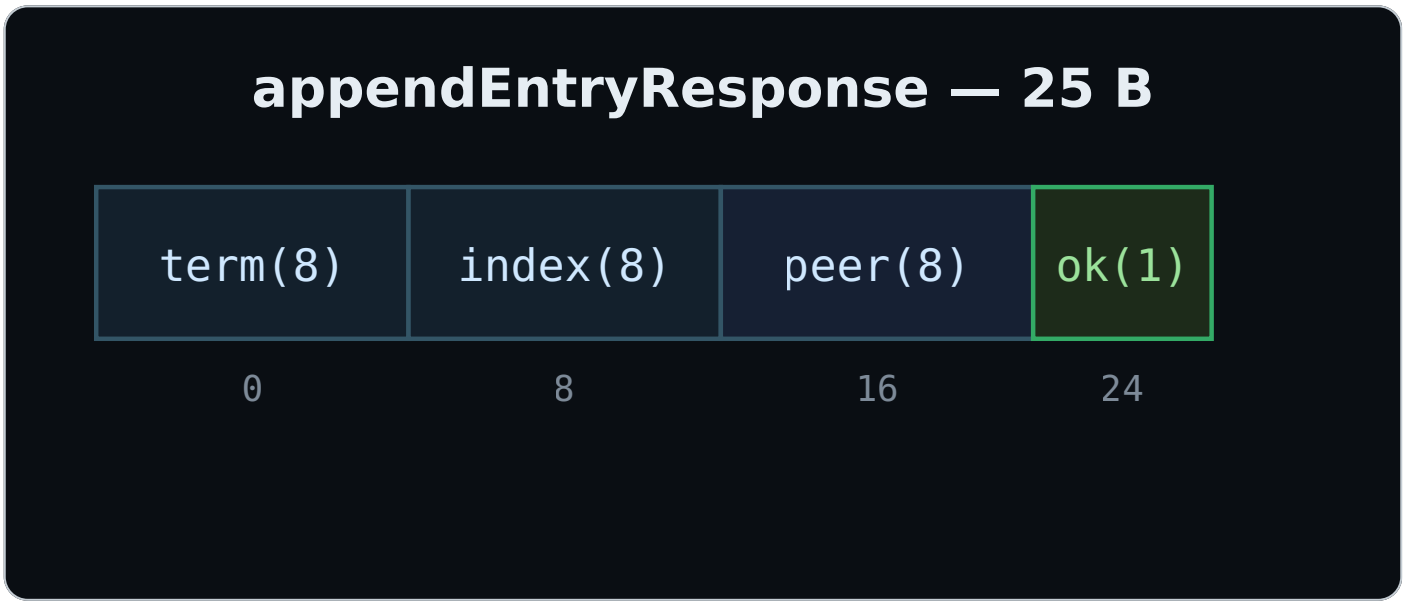
5 · Wire protocol — byte layouts

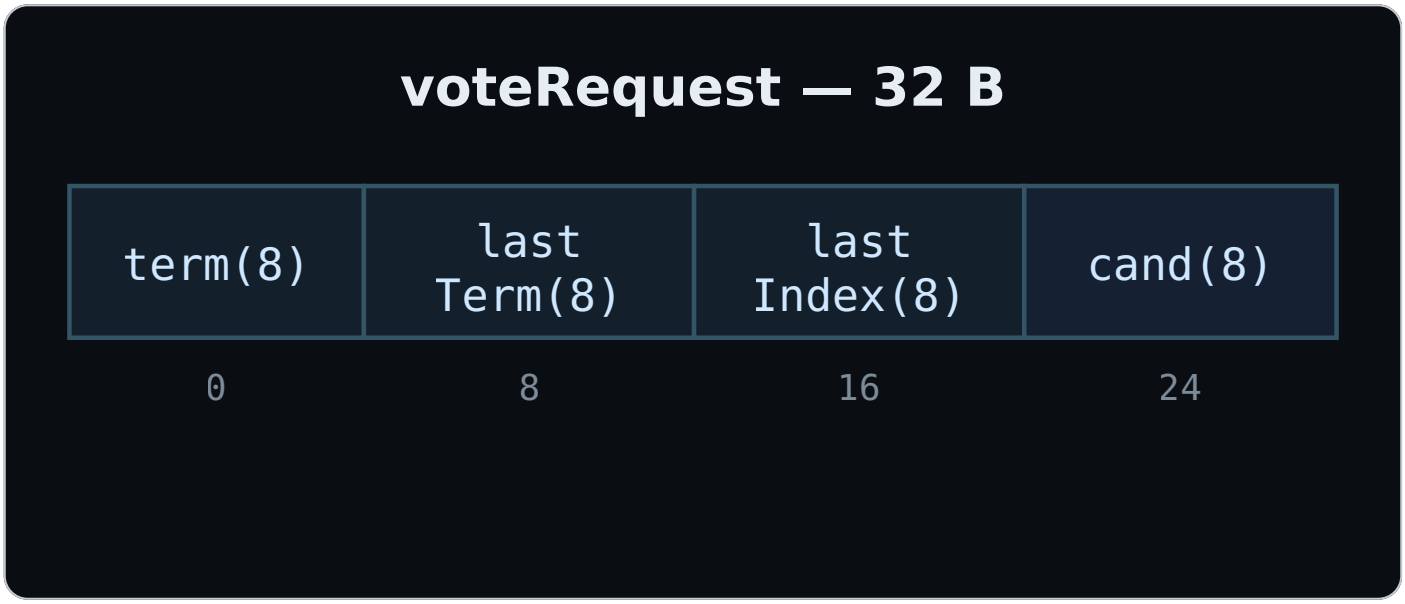
All four RPCs are hand-rolled, fixed/length-prefixed, **little-endian** binary. There is no protobuf/JSON on the hot path. Below, each cell is annotated with its byte offset.

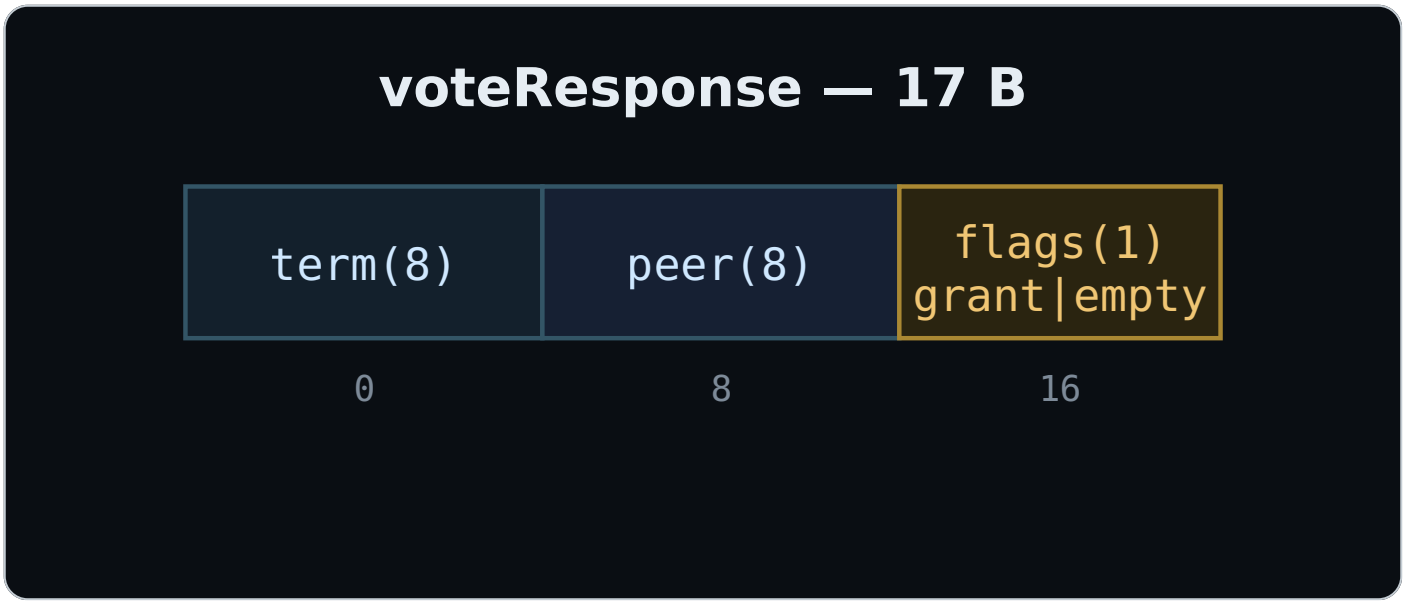
appendEntry `raft.go:2799 · base len = idLen + 4·8 + 2 = 50 B`



The trailing `lterm` (leader's highest term, used during catchup) is appended after the entries as a uvarint; older nodes safely ignore trailing bytes. `EntryType` values: `EntryNormal` (all app data), `EntryPeerState`, `EntryAddPeer`, `EntryRemovePeer`, `EntryLeaderTransfer`, `EntrySnapshot`, `EntryOldSnapshot`, plus the internal `EntryCatchup` `raft.go:2742-2756`.







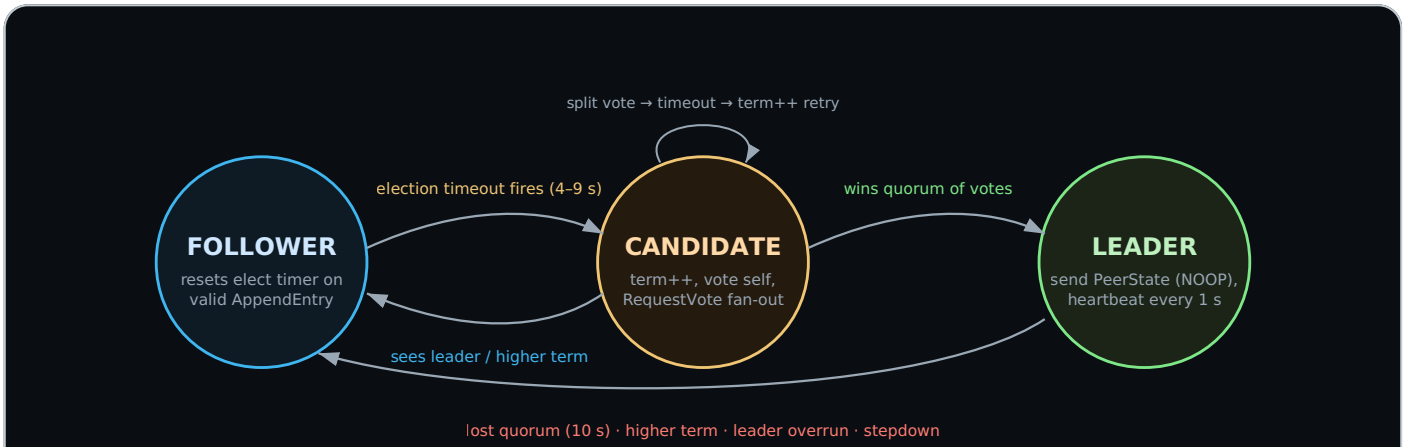
The vote response `flags` byte packs two bits: `granted` (bit 0) and `empty` (bit 1 — "my log is empty"). The `empty` bit drives a NATS-specific safety rule (see §5).

The application payload inside `EntryNormal` (a published message)

Raft is oblivious to JetStream semantics. A stream message is encoded by `encodeStreamMsg` into the `Entry.Data` bytes: `[op:1][lseq:8][ts:8][slen:2][subject][rlen:2][reply][hlen:2][hdr][mlen:4][msg][flags:uvarint]`, where `op = streamMsgOp` (or `compressedStreamMsgOp` when the total exceeds the 8 KB compress threshold) `jetstream_cluster.go:9893`.

6 · Leader election

The node is always in exactly one state, driven by a dedicated goroutine. Transitions follow the Raft paper closely, with a few deliberate NATS additions.



The standard three-state machine. Note the LEADER→FOLLOWER edge is taken proactively on *lost quorum* and *leader-overrun*, not only on seeing a higher term.

The vote decision

`processVoteRequest` `raft.go:5065` grants a vote only when (a) the candidate's term ≥ ours, (b) we haven't already voted for someone else this term, and (c) the candidate's log is at least as up-to-date (`lastTerm > pterm`, or equal term and `lastIndex ≥ pindex`) — exactly the Raft §5.4.1 "election restriction." Granting resets our election timer; if we have a *more* up-to-date log and haven't voted, we shorten our own timer to campaign sooner.

> **NATS-specific "empty log" rule.** A node with an empty log (`pindex == 0`) sets the `empty` bit in its vote response. A candidate that cannot reach a normal quorum may still win only if it has heard from **every** server and the rest came back empty `raft.go:3811-3831`. This guards against electing a leader on a fresh/empty log during scale-up — a real data-loss hazard that the vanilla paper does not address.

Prior art — how other systems avoid the empty-log hazard

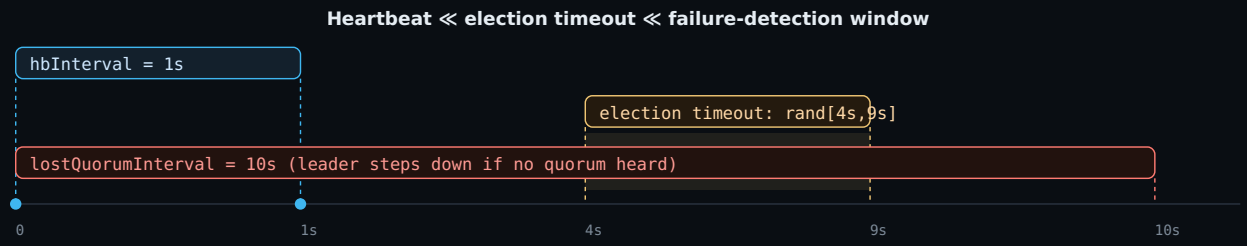
The *hazard* here is the classic **"catching up new servers"** problem (Raft thesis §4.2.1, §4.4): if the voter set grows with fresh, empty-log nodes, those nodes can form a quorum and elect a leader that is missing committed data → data loss or a stuck cluster. The plain election restriction (§5.4.1) isn't sufficient on its own during scale-up, because empty nodes compare "up-to-date" only against each other. The **goal is universal; the mainstream solution is structural — non-voting learners that must catch up before they can vote** — rather than a vote-time flag.

System	Mechanism
etcd	`Learner` members (since 3.4): docs prescribe adding a member as a learner, catching it up, then promoting to voter.
CockroachDB	`LEARNER` / non-voting replicas during rebalancing (etcd/raft based).
TiKV / raft-rs	`Learner` role — replicates but cannot vote until promoted.
Consul / HashiCorp Raft	**Non-voter** servers + Autopilot `ServerStabilizationTime`: a server stays non-voting until healthy/caught up.
MongoDB (Raft-like)	`newlyAdded` members are non-voting until initial sync completes, then auto-reconfigured to voting.
Kafka KRaft	Observers catch up before being added as voters (KIP-853 dynamic quorum).
Raft thesis	§4.2.1 — add new servers as non-voting, replicate until caught up, *then* count them.

> **Where NATS sits.** NATS *also* uses the learner approach — that is exactly what `Observer` mode + the `ScaleUp` flag are: a newly added peer can't campaign until its log has data (`raft.go` `scaleUp`/`setObserverLocked`). The empty-log vote bit is a complementary **vote-time** safety net on top of that: a would-be leader refuses to declare victory while it might be missing the one node that holds the only copy of committed data — i.e. it stays conservative precisely when it can't distinguish "I have the latest log" from "an unreachable peer has more." The exact wire mechanism (an `empty` bit + all-hands escalation) appears to be NATS-specific; most systems achieve the same safety structurally via learners.

Becoming leader is not instantaneous to the upper layer: `switchToLeader` immediately sends a PeerState entry (the implementation's equivalent of the Raft *no-op on election*) and defers signalling "I am leader" (`n.aflr`) until that entry — and everything before it — has been **applied**. This guarantees a new leader is caught up before it serves `raft.go:5304-5331`.

7 · Timers, intervals & heartbeats



The ratios satisfy the Raft paper's guidance (broadcast ≪ election timeout), but the absolute values are an order of magnitude larger than the paper's 150–300 ms — a deliberate trade favoring stability over a pub/sub transport.

Constant	Default	Role	`raft.go`
`minElectionTimeout` / `maxElectionTimeout`	**4 s / 9 s**	Randomized window a follower waits without contact before campaigning.	282-283
`minCampaignTimeout` / `maxCampaignTimeout`	100 ms / 800 ms	Short pre-campaign delay (e.g. preferred-leader transfer, faster start when log is ahead).	284-285
`hbInterval`	**1 s**	Leader heartbeat tick; a heartbeat is only sent if `notActive()` (no entry sent within the last 1 s).	286
`lostQuorumInterval`	10 s	A peer is "lost" if not heard from within this; a leader losing quorum steps down.	287
`lostQuorumCheck`	10 s	Ticker cadence for the leader's lost-quorum check.	288
peer-current (`hbInterval*2`)	2 s	A follower is considered "current" if seen within 2×hb.	1844, 2139
healthy (`hbInterval*3`)	3 s	`Healthy()` liveness threshold per peer.	1993, 2008
`observerModeInterval`	48 h	Observers never campaign — election timer parked.	289
`peerRemoveTimeout`	5 min	Grace before a removed peer can be re-tracked.	290
`pauseQuorumThreshold`	100,000	Uncommitted/unapplied backlog at which the leader self-steps-down (overrun).	4669
`paeDropThreshold`	20,000	In-memory pending-append-entry cache cap	4670

		before dropping (forces WAL reload on apply).	

> **Heartbeats are lazy and piggybacked.** Real data `appendEntry`s double as heartbeats. The dedicated `sendHeartbeat()` (an empty append entry) only fires when the leader has been idle > `hbInterval` `raft.go:3214-3216, 4816`. After a commit advances, the leader also pushes one extra heartbeat so followers can apply promptly without waiting a full tick `raft.go:4583-4585`.

8 · Replication & commit engine

The leader batches proposals (≤ 256 KB or ≤ 512 entries), writes them to **its own WAL first**, fans out, then commits on quorum. Two correctness rules from the paper are implemented precisely:

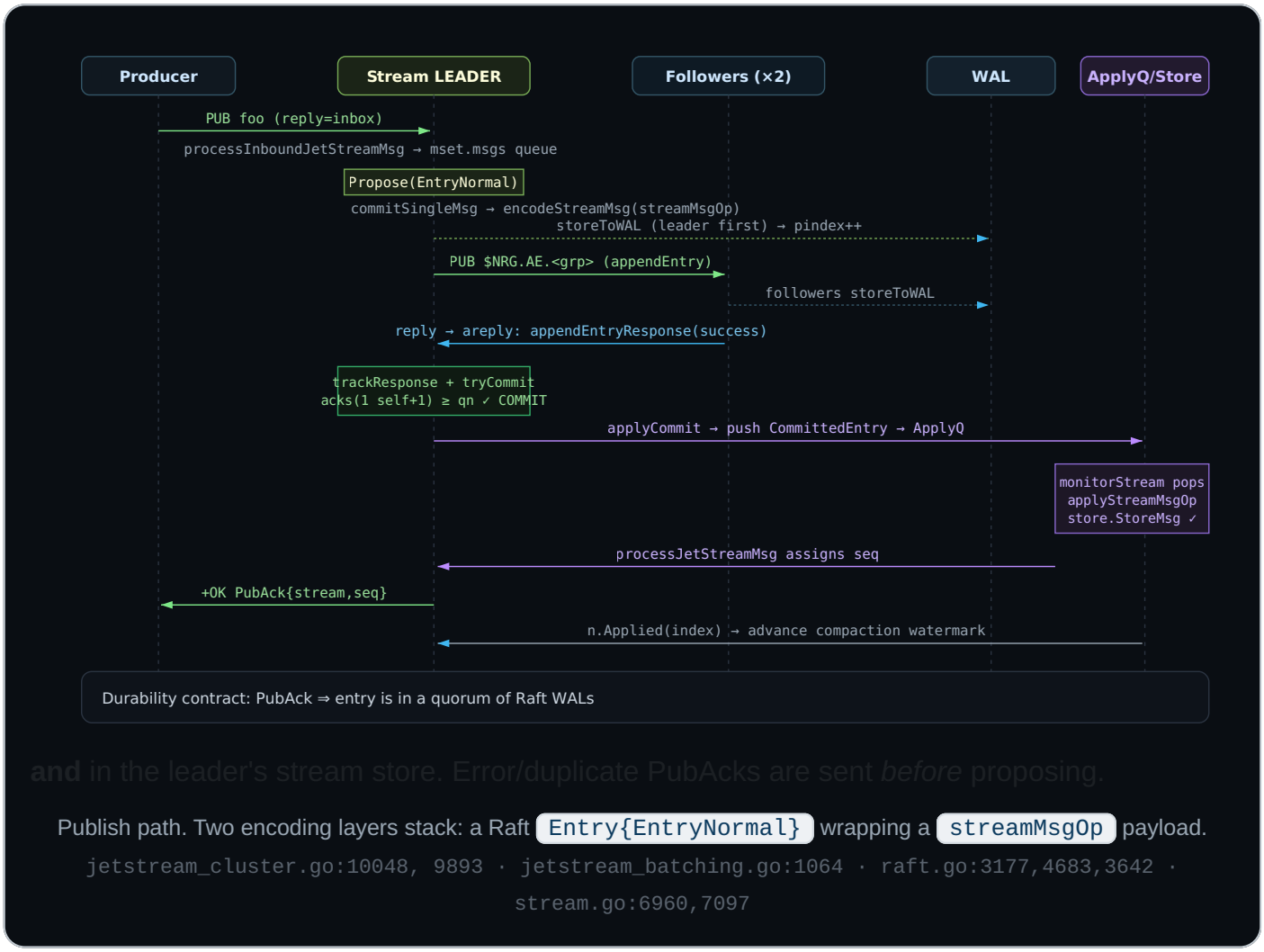
> **Quorum counts the leader.** `tryCommit` adds 1 for the leader itself (if still a member) and requires `acks ≥ qn = csz/2+1` `raft.go:3642-3661`.

> **Only current-term entries commit by counting.** `trackResponse` ignores acks where `ar.term != n.term` — earlier-term entries are committed only indirectly, matching Raft §5.4.2 / Figure 8 `raft.go:3686`.

Committed entries are pushed onto the **ApplyQ**; the upper layer (stream/consumer monitor) applies them and calls `Applied(index)` back, which advances the snapshot/compaction watermark and can release the new-leader latch.

9 · End-to-end: publishing a message

A producer publishes to a clustered (R>1) stream subject. The PubAck is returned only after the message is **committed (quorum-durable in Raft WALs)** and **applied (written to the leader's stream store)**.



and in the leader's stream store. Error/duplicate PubAcks are sent *before* proposing.

Publish path. Two encoding layers stack: a Raft `Entry{EntryNormal}` wrapping a `streamMsgOp` payload.

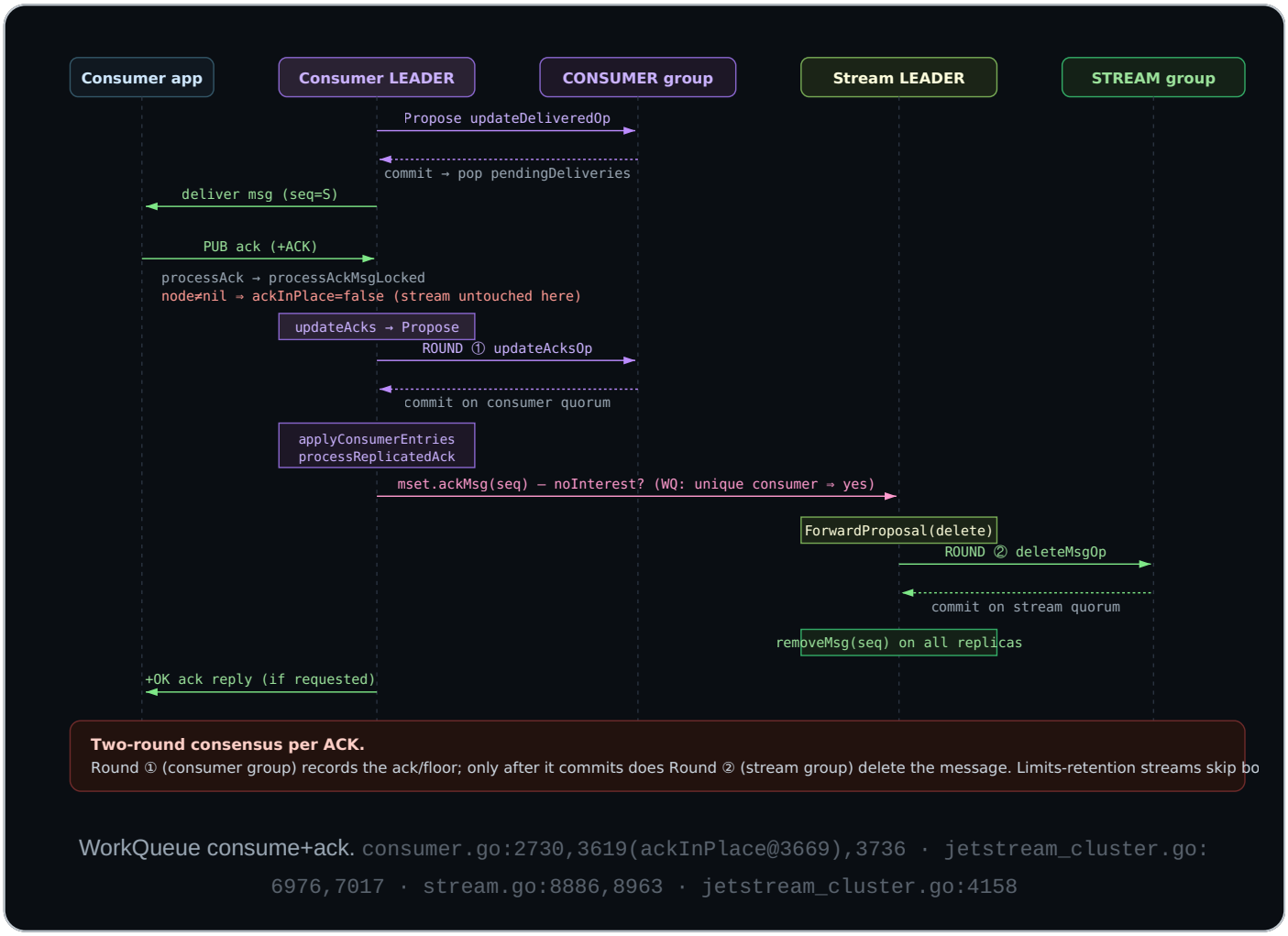
`jetstream_cluster.go:10048, 9893 · jetstream_batching.go:1064 · raft.go:3177,4683,3642 · stream.go:6960,7097`

10 · End-to-end: consume + acknowledge (WorkQueue)

WorkQueue retention deletes a message once it is acknowledged. On a clustered stream this is the most consensus-heavy flow in the system: **a single client ACK drives two serialized Raft rounds** — first on the consumer group (record the ack), then on the stream group (delete the message).

Delivery is also quorum-gated

For a clustered consumer with an ack policy other than `AckNone`, `deliverMsg` first proposes `updateDeliveredOp` and parks the message in `pendingDeliveries`; the client only receives it after that delivered-state commits `consumer.go:5645-5688 · jetstream_cluster.go:6858-6873`. This prevents a just-failed-over leader from re-delivering inconsistently.



WorkQueue consume+ack. `consumer.go:2730,3619(ackInPlace@3669),3736 · jetstream_cluster.go:6976,7017 · stream.go:8886,8963 · jetstream_cluster.go:4158`

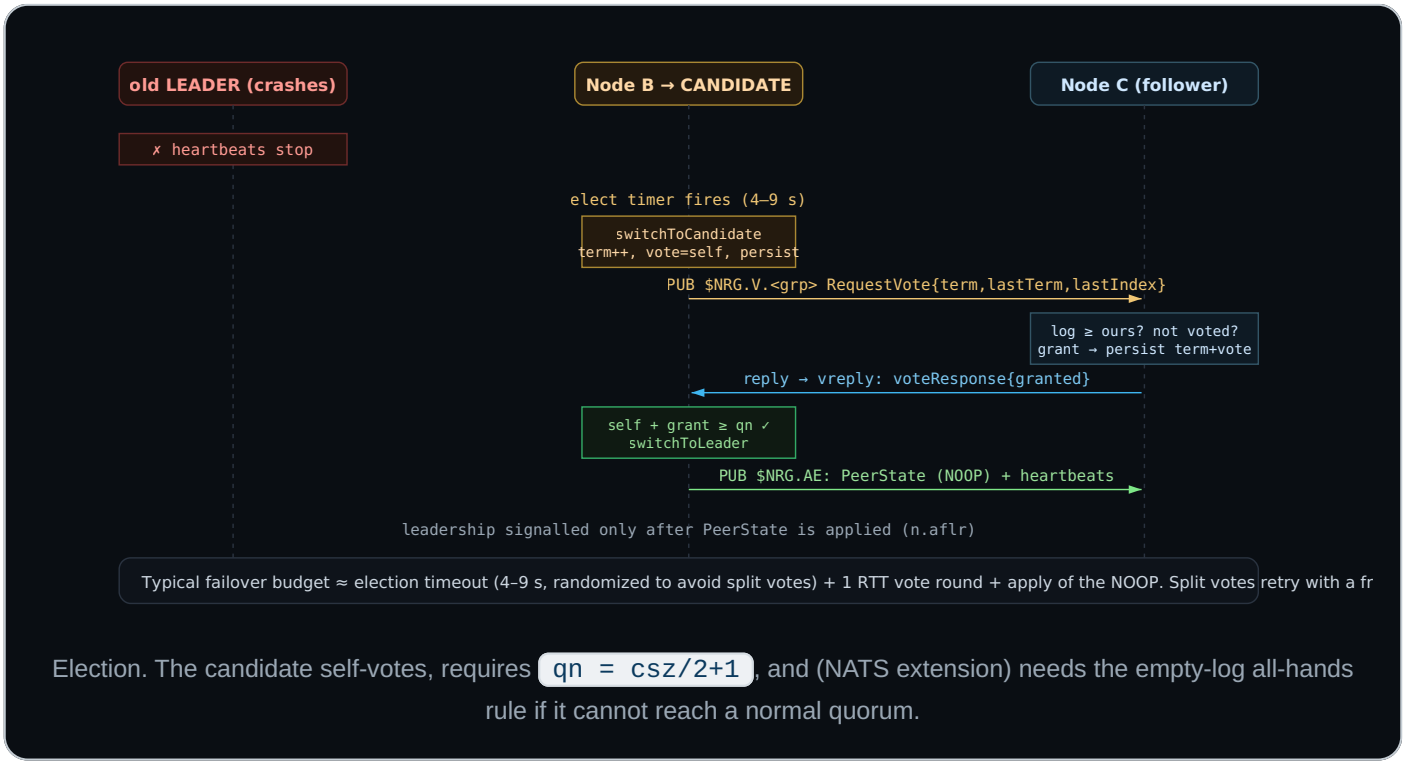
Consumer Raft entry ops & ack policies

Op	Meaning
`updateDeliveredOp`	message delivered (dseq,sseq,dc,ts)
`updateAcksOp`	message acked (dseq,sseq)
`updateSkipOp`	filtered consumer skipped ahead
`addPendingRequest` / `removePendingRequest`	track in-flight pull waiters
`resetSeqOp`	reset start sequence (quorum-gated)

Ack policy → Raft cost

- **AckExplicit** — one `updateAcksOp` *per message*; `Pending` / `Redelivered` maps replicated. WorkQueue forces this.
- **AckAll** — one op covers a whole range; apply reconstructs the gap and removes each seq. Fewer proposals.
- **AckNone** — no client acks; delivery *not* quorum-gated; removal driven from `deliverMsg` (still an `updateAcksOp` for clustered non-direct).

11 · End-to-end: a leader election



Election. The candidate self-votes, requires `qn = csz/2+1`, and (NATS extension) needs the empty-log all-hands rule if it cannot reach a normal quorum.

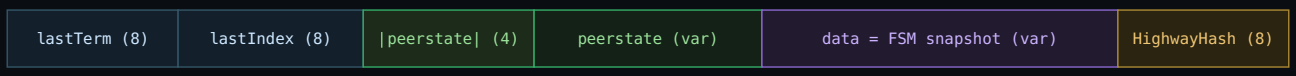
12 · Snapshots, compaction & catchup

A Raft log cannot grow forever, so each layer periodically captures its **application state** as a snapshot and **compacts** (truncates) the WAL behind it. The crucial design point in NATS: the snapshot is **produced by the upper layer** (the FSM), not by Raft. Raft only stores the snapshot blob + a watermark and discards superseded log entries. Two distinct words matter:

Snapshot = an opaque, self-contained encoding of the FSM's current state (meta assignments / stream state / consumer ack state) at a given applied index. Stored at `<sd>/snapshots/snap.<term>.<index>`.

Compaction = `wal.Compact(snapIndex+1)` — physically drops all log entries ≤ the snapshot index, which is what actually reclaims disk/memory `raft.go:1381`.

The snapshot file format



`encodeSnapshot` `raft.go:1296`: a 28-byte header+checksum frame (`minSnapshotLen`) wrapping the peer state and the upper-layer blob; integrity guarded by a trailing HighwayHash-64.

How a snapshot is produced — the checkpoint dance

To avoid blocking the Raft node while the FSM serializes potentially large state, snapshotting defaults to **asynchronous** via a *checkpoint* handle (`CreateSnapshotCheckpoint` → `RaftNodeCheckpoint`) `raft.go:1403`. The upper layer can build the snapshot off-lock, then atomically install it; it may also `Abort()` at any time.

blocking snapshot after repeated async failures or when the log grows past 10× the size threshold.

Async snapshot via the checkpoint API. The Raft lock is held only briefly to pin the index and to install; the expensive FSM encoding happens unlocked. `raft.go:1337-1520 · jetstream_cluster.go:1636-1797`

Triggers & timers per layer

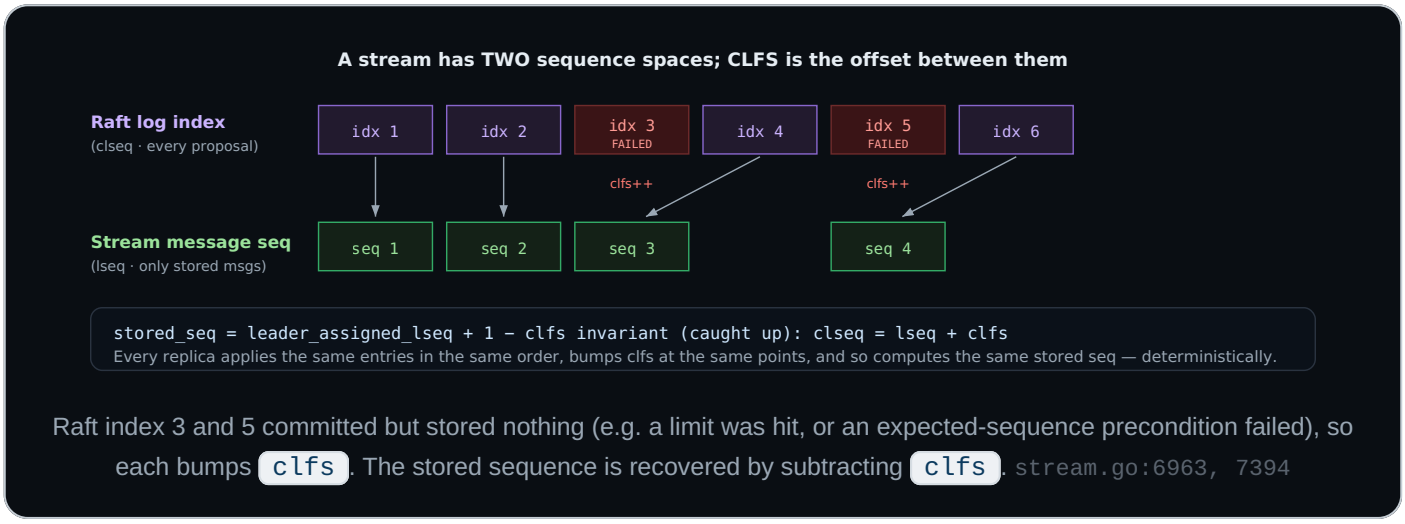
Each FSM monitor runs its own snapshot timer plus event-driven checks after every batch of applied entries. `Applied()` returns the count of entries / bytes now reclaimable, which feeds the threshold checks.

Layer	Periodic timer	Min interval after failure	Min delta between snaps	Size / entry thresholds	`jetstream_cluster.go`
Meta	`compactInterval = 1 min`	`compactMinInterval = 15 s` (timer reset on failure)	`minSnapDelta = 30 s`	> 8 MB (or configurable `JetStreamMetaCompact[Size]`), or `NeedSnapshot()`; peer-entries force immediately	1588-1615
Stream	`2 min + rand(0–1 min)` jitter	`15 s`	—	≥ 65,536 entries, or > 8 MB, or CLFS advanced	3119-3123, 3446
Consumer	`2 min + rand(0–1 min)` jitter	`15 s`	`minSnapDelta = 10 s`	≥ 1,024 entries, or > 64 KB	6508-6513, 6627

- **Startup jitter** (stream/consumer): a random 0–60 s is added to the 2-min ticker so thousands of groups don't all snapshot in lockstep after a restart `jetstream_cluster.go:3126`.
- **Forced snapshots**: taken on becoming leader, right after recovery (to start with a small log), and on membership (peer) changes; meta forces if > 4 async failures accumulate.
- **Failure back-off**: a failed async snapshot reduces the timer to 15 s and switches the next attempt to blocking, so the log cannot grow unbounded while async keeps failing.

CLFS — reconciling the two sequence spaces

CLFS = "Count of (the number of) Failed NRG Sequences" (`NRG` = NATS Raft Group). It is the running count of committed Raft log entries on a stream's group that **did not produce a stored stream message** `stream.go:554`. It exists to bridge a fundamental mismatch in clustered streams:



Raft index 3 and 5 committed but stored nothing (e.g. a limit was hit, or an expected-sequence precondition failed), so each bumps `clfs`. The stored sequence is recovered by subtracting `clfs`. `stream.go:6963, 7394`

Why the two spaces diverge

The leader proposes optimistically and bumps `clseq` (last seq sent to the NRG layer) immediately, so it can pipeline ahead of commits. But not every committed entry results in a stored message. On apply, `processJetStreamMsg` calls `bumpCLFS()` when an entry is deterministically rejected:

- store limit hit — `MaxMsgs` / `MaxBytes` / `MaxMsgsPerSubject` / `MsgTooLarge` `stream.go:6990`
- `expected-last-sequence` or `expected-last-msgId` precondition mismatch `stream.go:6627`
- rejected partial-batch entries

Pure I/O errors do *not* bump CLFS — they stop the replica instead, so state stays comparable.

Why it must be consistent — and why it gates snapshots

- **Determinism:** because every replica must compute the *same* stored sequence, all replicas must agree on exactly which entries "failed." This is why a partial batch is explicitly rejected on every node "to ensure CLFS is correct" `jetstream_cluster.go:3377`.
- **Persisted in the snapshot:** CLFS is part of the stream snapshot (`streamSnapshot.Failed` / `EncodedStreamState(getCLFS())`) `jetstream_cluster.go:9999, 10016`, so a recovering or caught-up node restores the correct offset.
- **Snapshot trigger:** the monitor records `pclfs` before applying a batch and snapshots if `getCLFS() > pclfs` — i.e. whenever this batch advanced CLFS — so the new offset is durably captured promptly (within the min-delta window) `jetstream_cluster.go:3353, 3446`.

Operational signal. A persistently climbing CLFS means the stream is committing Raft entries that store nothing — typically a client repeatedly hitting limits or sending bad expected-sequence/msgId preconditions. It inflates the Raft log relative to stored data and triggers extra snapshots. The lag check `clseq − (lseq + clfs) > streamLagWarnThreshold` emits a "high message lag" warning when the leader runs too far ahead of applies `stream.go:7531`.

Installing a leader snapshot & catchup

A follower that is too far behind (below the leader's first WAL sequence, e.g. just placed via scale-up) cannot be caught up entry-by-entry, so it receives a **snapshot instead**.

Catchup flow: a behind follower replies *unsuccessfully* to an append entry, putting a fresh `$NRG.CR.<rnd>` catchup inbox in the reply subject. The leader runs `catchupFollower` → `runCatchup`, streaming WAL entries to that inbox with a **2 MB outstanding window**; if the follower is below the WAL's first sequence it calls `sendSnapshotToFollower`, shipping the snapshot as an `EntrySnapshot` + `EntryPeerState` append entry `raft.go:3306,3413,3440`. There is **no dedicated snapshot subject** — snapshots ride the normal append-entry channel. Catchup responses *never* count toward quorum (a key safety guard), and the leader refuses to create a new snapshot while a catchup is in progress unless forced.

On the receiving side, an `EntrySnapshot` commit triggers `installSnapshot`: write the snap file, `wal.Compact`, advance `papplied`, and hand the blob up to the FSM via the ApplyQ so it can rebuild its state `raft.go:1357, 3581-3591`.

13 · Subscription budget (capacity planning)

Subscription count scales with the number of Raft groups, which scales with streams × replicas and consumers × replicas. This is a real operational dimension on dense servers.

4

subs per group on a follower (AE, V, areply, vreply)

6

subs per group on a leader (+ P, RP)

+1

transient catchup sub while behind

Per server, total ≈ Σ over (meta + each hosted stream replica + each hosted consumer replica) of 4–6. A server hosting 1,000 R3 streams each with one R3 consumer holds on the order of 1,000×~5 + 1,000×~5 + meta ≈ **~10k Raft subscriptions**, all plain (non-queue) on the internal system client.

14 · Risks & analysis vs. the Raft paper and field implementations

Compared against Ongaro & Ousterhout's Raft (the 2014 paper and Ongaro's PhD thesis) and mature implementations (etcd/raft, HashiCorp Raft, Dragonboat), the design is **safety-faithful in its core** but makes transport- and workload-specific choices that carry trade-offs.

#	Finding	Severity	Detail & comparison
R1	No Pre-Vote phase `prototype added`	`High`	`switchToCandidate` increments the term *before* soliciting votes `raft.go:5292`. A node partitioned away (or flapping) rejoins with an inflated term and forces the healthy leader to step down — the classic "disruptive server" problem. Ongaro's thesis §9.6 prescribes **PreVote** (campaign in a trial term, only bump the real term once a majority would grant). etcd/raft and HashiCorp Raft both ship PreVote. NATS partially mitigates via the large 4–9 s timeout and the empty-log rule, but the core protection is absent. **A working prototype is implemented on this branch — see §15.**
R2	Coarse failure detection / stale-leader window	`Medium`	CheckQuorum is implemented as a 10 s `lostQuorumInterval` / `lostQuorumCheck` `raft.go:287-288,3258`. A partitioned leader can believe it is leader for up to ~10 s. Writes still fail safely (they need quorum acks), but anything keyed off `Leader()` / `Current()` for routing or non-linearizable reads can be stale for seconds. etcd uses a leader lease tied to the (sub-second) election timeout. **Recommendation:**

			ensure all read/routing paths gate on quorum freshness, not just the leader flag.
R3	Multi-second failover by design	Medium	Election timeout 4–9 s with a 1 s heartbeat means leader loss is detected only after several seconds, and a contended election adds more. The paper targets 150–300 ms. This is an intentional trade for stability over a pub/sub transport and WAN/ gossip clusters, but applications must tolerate multi-second unavailability windows per group during failover. Many groups failing over together (e.g. node restart) amplifies the storm.
R4	Double-consensus for WorkQueue ack	Medium	One client ACK = two serialized Raft rounds (consumer group, then stream group; §9). This is *correct* and deliberately ordered, but it doubles latency and write-amplification on the hottest path, and creates a window where the ack is committed but the delete is not. Idempotency (pre-acks, dedupe, redelivery-on-recover) covers it, but exactly-once consumers must understand that a crash between rounds can surface as a redelivery. Contrast: a single-log design

			(consumer+stream colocated) would need one round, at the cost of coupling.
R5	Lossy transport, limited per-peer flow control	Info	RPCs ride core NATS (at-most-once); the only outbound primitive is fire-and-forget `sendRPC` `raft.go:5159`. Raft tolerates loss by design (heartbeat/catchup recover), so this is safe, but there is no per-peer pipelining/flow-window like etcd's MsgApp (catchup has its own 2 MB window). On a saturated system, dropped append entries simply convert into catchups. Acceptable, but capacity-sensitive.
R6	Leader-overrun self-stepdown can cause churn	Medium	A leader that accrues > 100,000 uncommitted/unapplied entries steps down `raft.go:974-984`. Under sustained overload or a slow follower, the most up-to-date node abdicates, triggering an election; the replacement may hit the same wall → election churn. The intent ("let the system breathe") is sound, but it couples back-pressure to leadership stability. **Recommendation:** monitor `overrunCount`; prefer client-side back-pressure before the threshold.
R7		Low	

	Forwarded proposals are unconfirmed		`ForwardProposal` sends to `$NRG.P` with no reply and no ack — there's an explicit TODO to add confirmation `raft.go:988-998`. In JetStream the leader is the proposer so impact is limited, but any path relying on follower-side forwarding gets no delivery/commit guarantee.
R8	Pending-entry cache eviction → WAL reload	`Low`	Above `paeDropThreshold` (20,000) the in-memory append-entry cache is bypassed `raft.go:4718-4732`, so `applyCommit` re-reads from the WAL — extra I/O exactly when the node is already behind. Self-limiting but worth watching under burst.
R9	Fail-stop on any write error	`Info`	A critical WAL write error sets `werr` and **shuts the node down** `raft.go:4929-4965`. Excellent for safety (no silent corruption), but a transient disk/quota hiccup takes the replica offline until restart — a deliberate availability-for-safety trade. Out-of-space/permission errors escalate to disabling JetStream.
R10	8-byte (truncated-hash) node IDs	`Info`	`idLen = 8` from a HighwayHash of the server name `raft.go:2893`. Collision probability is negligible at realistic

			cluster sizes (birthday bound), but it is a truncated hash rather than a UUID; operationally, ensure server names are unique.
R11	Full (non-incremental) meta snapshot cost	Medium	Every meta snapshot re-serializes the **entire** cluster config — all streams + consumers in all accounts — via JSON-marshal + S2-compress, with no delta encoding (§3; `encodeMetaSnapshot` `jetstream_cluster.go:2195`). Cost scales with total cluster size, not churn, so on large clusters (10k+ assets) snapshots become slow and the `_meta_` WAL can only compact as fast as they finish. The async path keeps it off `js.mu`, but the **blocking fallback holds the JS read lock for the whole serialization** `jetstream_cluster.go:2010`, stalling all control-plane operations cluster-wide for its duration. **Recommendation:** alert on `lastMetaSnapDuration` / "snapshot generation took" warnings; keep total asset counts within tested bounds; avoid config churn storms that force frequent meta snapshots.

What the implementation gets right (highlights)

> **Figure-8 safety.** Commits only count acks for current-term entries (`ar.term == n.term`), so a new leader never commits a prior-term entry by raw vote-counting `raft.go:3686` — the subtle rule many home-grown Rafts miss.

> **Election restriction.** Votes require an equal-or-more-up-to-date log (lastTerm/lastIndex), preventing a lagging node from winning `raft.go:5109`.

> **Empty-log guard.** The `empty` vote bit + all-hands rule closes a real scale-up data-loss hole beyond the vanilla paper `raft.go:3811-3831`.

> **Single-server membership changes.** One change at a time, serialized via `membChangeIndex` — the simpler, safe approach from Raft §4 (avoids joint-consensus complexity).

> **Caught-up before serving.** A new leader withholds the leadership signal until its election NOOP (PeerState) is applied (`n.aflr`) — avoids serving stale state right after winning.

> **Durable PubAck contract.** Acks to publishers are sent only after quorum-commit *and* leader-store apply — a clear, strong durability guarantee.

15 · Pre-Vote phase (prototype)

This section documents the Pre-Vote prototype implemented on this branch to address risk R1. Pre-Vote inserts a **trial election that does not touch the term**: a node first asks "would a real election at `term+1` succeed?" and only escalates — incrementing its term and running a normal election — if a quorum would grant. The decisive rule lives on the **voter**: grant a pre-vote only if you are *not* currently hearing from a leader.

Prior art — pre-vote in other systems

Pre-Vote is a well-established Raft extension; most mature implementations ship it. NATS not having it until now is not unusual — it lives in **Ongaro's PhD thesis §9.6** ("preventing disruptions

when a server rejoins"), not the short extended paper, so from-scratch implementations often defer it until partition/flap churn shows up at scale.

System / library	Pre-Vote?	Notes
etcd/raft (Go)	`Yes`	The reference implementation: explicit `PreVote` option and a `StatePreCandidate` state (`campaignPreElection`). The design this prototype most closely mirrors. Configurable since etcd **3.4** (`--pre-vote`); default-on in recent releases.
CockroachDB	`Yes`	Uses a fork of etcd/raft, so inherits pre-vote (paired with quiesced ranges + CheckQuorum).
TiKV / raft-rs (Rust)	`Yes`	raft-rs is a Rust port of etcd/raft; pre-vote is first-class and TiKV runs with it.
HashiCorp Raft (Consul, Nomad, Vault)	`Yes (recent)`	Long-requested (PR #530 sat for years); landed in **hashicorp/raft v1.7.0 (2024)**. Earlier releases relied on CheckQuorum + leadership transfer + non-voter staging instead. Availability in a given Consul/Vault build depends on the vendored raft-lib version.
Apache Ratis (Java; Apache Ozone)	`Yes`	Has a pre-vote phase.
Dragonboat (Go, multi-group)	`Not classic form`	Performance-focused; historically leaned on other mechanisms (observers/witnesses, CheckQuorum-style) rather than the etcd-style pre-vote state.
Raft thesis / textbook Raft	`Extension`	Defined in Ongaro thesis §9.6 — an optional extension, not part of the core figure-2 protocol.

Pre-Vote is usually paired with CheckQuorum, not a substitute for it. etcd runs both: *PreVote* stops a returning node from disrupting a healthy leader, while *CheckQuorum* makes a partitioned leader step down so a genuine failover can proceed. NATS already has the CheckQuorum half (the 10 s `lostQuorumInterval` stepdown); this prototype adds the PreVote half, bringing it in line with the etcd model. The thesis actually proposes two complementary disruption defenses — the trial election *and* "ignore a RequestVote received within the minimum election timeout of hearing from a leader"; etcd implements both, and our voter guard (`recentLeaderContactLocked`) folds the second into the pre-vote grant decision.

Scenarios it solves

1 · Disruptive rejoin after a partition

A node isolated from the cluster keeps timing out and, in plain Raft, increments its term on each attempt. When the partition heals it rejoins with a much higher term; its first vote/append-entry forces the healthy leader to step down (`processVoteRequest` steps down on higher term), causing a needless election — even though the rejoiner's log may be too stale to win. **With Pre-Vote** the healthy followers refuse the pre-vote (they have a leader), so the rejoiner never bumps its term and never disrupts.

2 · A flapping / half-partitioned node

A node whose link drops heartbeats intermittently (or can send but not receive) repeatedly campaigns and preempts the leader. Pre-Vote turns each of these into a harmless, term-preserving trial that fails fast against a live leader — no term inflation, no churn.

3 · Term inflation across long outages

Without Pre-Vote, terms ratchet upward for every failed election during a long isolation; the eventual rejoin causes a large term jump and repeated leader changes. Pre-Vote keeps the term pinned until a win is actually achievable.

4 · Election storms at scale

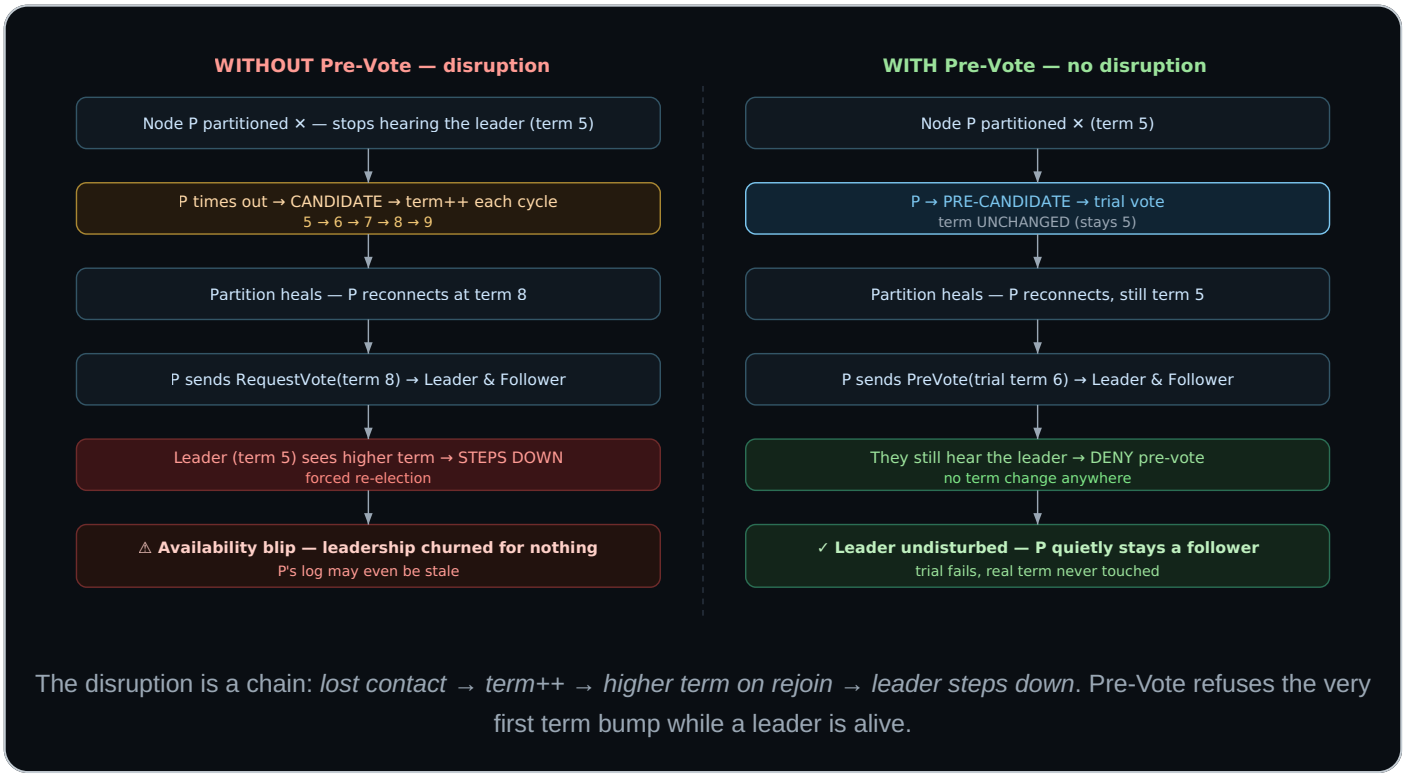
With one Raft group *per stream and per consumer*, a single misbehaving server can trigger simultaneous re-elections across thousands of groups, each a multi-second outage at NATS's 4–9 s timeout. Pre-Vote suppresses the spurious campaigns at the source.

What it preserves (liveness). When the leader genuinely fails, every follower's "last leader contact" ages out, so they all grant pre-votes and the trial round succeeds — escalating to a normal election. Failover still works; only *spurious* campaigns are blocked.

How each failure happens — before / after

① Disruptive rejoin

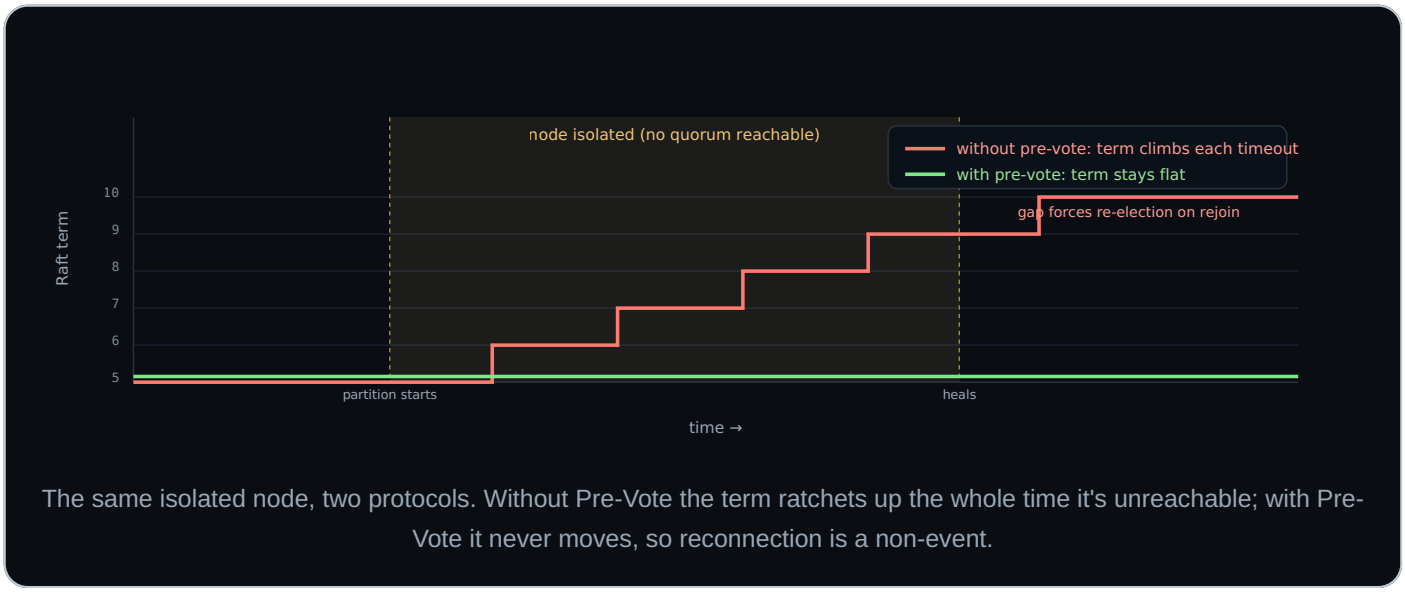
This is the headline failure (risk R1). In plain Raft, the only way a follower reacts to "I haven't heard from the leader" is to **increment its term and campaign**. A node that was merely *partitioned* (its link dropped, but it and the real leader are both perfectly healthy) does exactly that — repeatedly. When the link comes back, the node re-joins carrying a term far higher than the cluster's. Raft's core rule "on seeing a higher term, step down" then forces the healthy leader to abdicate, even though the rejoining node may have a stale log and no business leading. The result is a needless election and an availability blip. Pre-Vote breaks the chain at its first link: the rejoining node asks permission *before* bumping its term, and a cluster that still has a leader says no.



The disruption is a chain: *lost contact → term++ → higher term on rejoin → leader steps down*. Pre-Vote refuses the very first term bump while a leader is alive.

② Term inflation

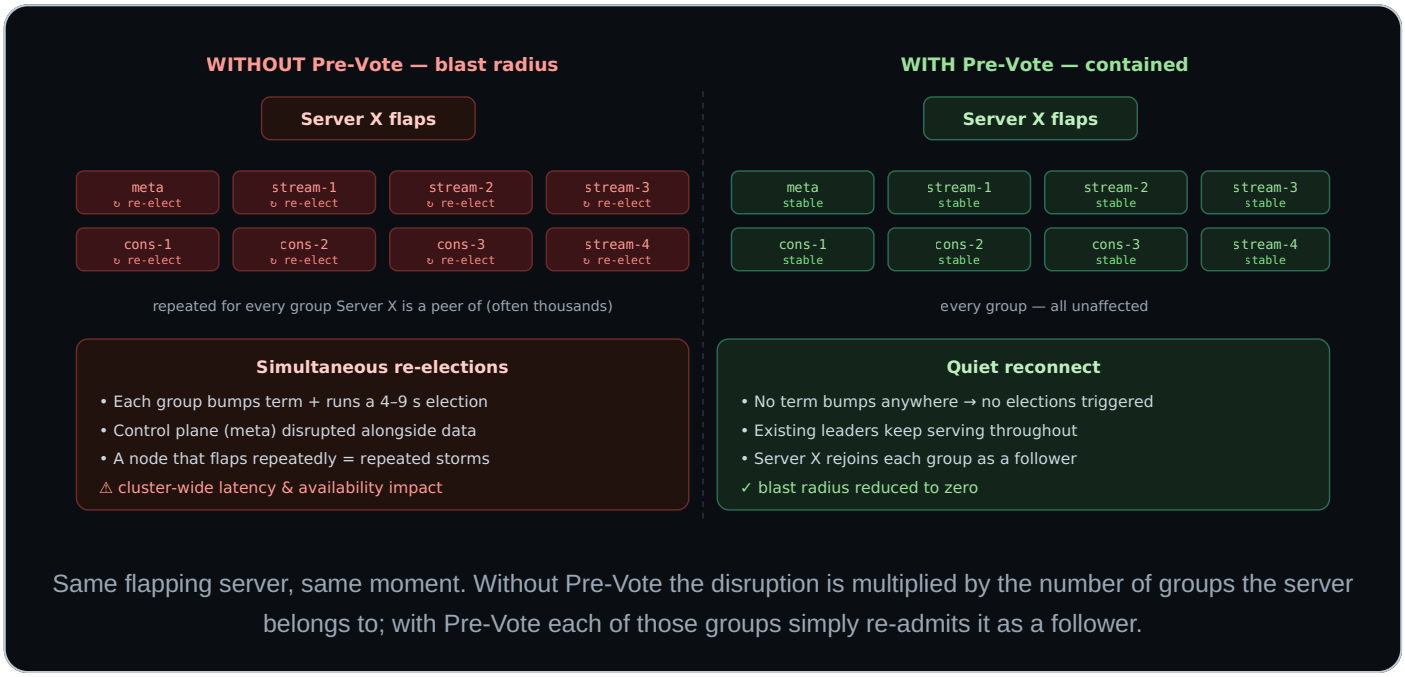
Term inflation is the *mechanism* behind scenario ①, worth seeing on its own. Each failed election attempt during an outage increments the term, and terms in Raft only ever go up. A node isolated for a long time (a flapping NIC, a one-way network fault, a stuck route) can rack up a large term gap. The bigger the gap, the bigger the disruption when it returns — and a node oscillating in and out can do this indefinitely. With Pre-Vote, an unreachable node spins on harmless trial rounds that never touch its term, so the gap stays at zero no matter how long the outage lasts.



The same isolated node, two protocols. Without Pre-Vote the term ratchets up the whole time it's unreachable; with Pre-Vote it never moves, so reconnection is a non-event.

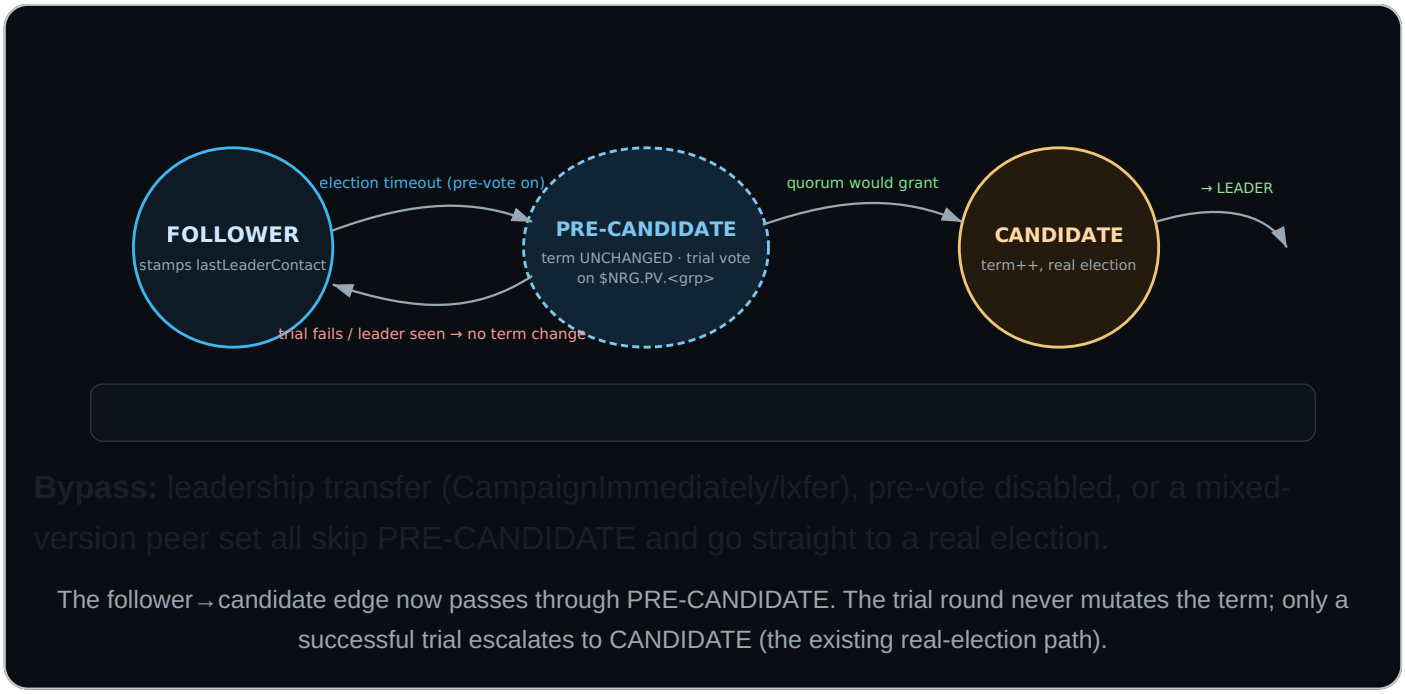
③ Cluster-wide re-election storm

NATS runs **one Raft group per stream and per consumer**, plus the meta group — a single dense server can be a peer in *thousands* of groups at once. A disruptive event on that one server (a restart, a brief partition, a flapping link) therefore doesn't disrupt one group — it disrupts *every group it participates in, simultaneously*. Each of those groups pays the multi-second election cost (4–9 s timeout) at the same moment, and the meta/control-plane group is in the blast radius too. Because Pre-Vote stops the term bump at the source, the same event becomes a quiet reconnect across all of those groups.



Same flapping server, same moment. Without Pre-Vote the disruption is multiplied by the number of groups the server belongs to; with Pre-Vote each of those groups simply re-admits it as a follower.

State machine



The follower→candidate edge now passes through PRE-CANDIDATE. The trial round never mutates the term; only a successful trial escalates to CANDIDATE (the existing real-election path).

How the prototype is wired (this branch)

- **State:** new `PreCandidate RaftState`; dispatched by `run()` to a new `runAsPreCandidate` loop.
- **Entry point:** the follower's election-timeout now calls `switchToPreCandidate` (no `term++`) instead of `switchToCandidate`. On a successful trial it calls the *unchanged* `switchToCandidate` → `requestVote`.

- **Wire:** reuses the `voteRequest`/`voteResponse` bytes verbatim on a new `$NRG.PV.<group>` subject + a private `pvreply` inbox — so the format stays backwards compatible (old peers don't subscribe). The voter responds with the *prospective* term so grants match.
- **Voter rule (`grantPreVoteLocked` / `processPreVoteRequest`):** mutates nothing; grants only if the log is up-to-date **and** there is no recent leader contact. The guard `recentLeaderContactLocked` refuses while a leader looks alive: it returns true if **we are the leader** (a leader never helps unseat itself) or if we accepted a leader append entry/heartbeat within `preVoteLeaderContactInterval` (a named constant = 2×`hbInterval`). `lastLeaderContact` is stamped whenever a follower accepts a leader's append entry/heartbeat.
- **Safety gates:** skipped for leadership transfer (`lxfer`), when disabled (`RaftConfig.PreVote`), or when `allPeersSupportPreVote()` reports a peer that does not advertise the capability — falling back to today's behavior so liveness is never lost during a rolling upgrade.
- **Capability negotiation:** a dedicated `PreVote` `ServerCapability` flag is advertised in statsz (`SetPreVote`) and recorded per peer as `nodeInfo.prevote` — the same mechanism as `BinaryStreamSnapshot`/`AccountNRG`, not a version string.
- **Liveness parity:** a repeatedly-failing trial round still surfaces the lost-quorum alert to the upper layer (mirrors the real candidate's behavior) on timeout.
- **Default:** enabled for JetStream stream/consumer groups via `createRaftGroup` and for the **meta (control-plane) group** — where a flapping server disrupting the meta leader is most costly — both with `PreVote: true`.

Test coverage added

Test	What it asserts
`TestNRGPreVotePreventsLeaderDisruption` `headline`	Drives the **same** election-timeout entry point through two sub-tests on independent groups: **WithoutPreVote** the follower bumps its term and the cluster term advances (disruption); **WithPreVote** the trial is refused while the leader is alive, so no term is bumped and the leader is unchanged. This is the direct before/after demonstration of the fix.
`TestNRGPreVoteDoesNotDisruptHealthyLeader`	An isolated node broadcasting a pre-vote at `term+100` against a healthy cluster changes **no node's term** and the original leader keeps leading — the core anti-disruption property (mirror-image of the existing real-vote isolation test, which asserts the cluster *does* jump term).
`TestNRGPreVoteElectsNewLeaderWhenLeaderStops`	With pre-vote enabled, stopping the leader still elects a new leader at a higher term — confirming the trial round escalates and failover is not broken.
`TestNRGPreVoteObserverDoesNotPreCampaign`	An observer never enters PreCandidate/Candidate and never bumps its term, even when driven down the election-timeout path.
`TestNRGPreVoteLeaderTransferBypassesPreVote`	Leadership transfer skips the trial round and the target becomes leader promptly (would hang if the bypass regressed).
`TestNRGPreVoteFlappingNodeNeverDisrupts`	A node forced into the trial 25× in a row never disrupts the healthy leader (term/leader stable), and the cluster still commits a proposal afterwards.
`TestNRGPreVoteGrantDecision`	Deterministic unit coverage of the voter guard: no-leader grant, leader self-refusal, recent vs aged leader contact, behind-log denial, and the report-higher-term back-off.
`TestNRGPreVoteEscalationTally`	Deterministic unit coverage of the escalation predicate `wonPreVote`, including the **all-hands/empty-log rule** that preserves scale-up protection in the trial round.
`TestNRGPreVotePartitionSoakDoesNotDisruptMeta` `real partition`	Builds a 3-node cluster where one server reaches the others *only* through two stoppable `netProxy` links (with `no_advertise` so route gossip can't bypass them); cutting them is a genuine network partition. It measures the isolated node's own meta term:

	WithPreVote it never inflates (Δ=0) so there is nothing to disrupt with on rejoin, while **WithoutPreVote it climbs** — proving both that the partition is real and that Pre-Vote is what keeps the term pinned. Lives in the `norace` bucket (fixed ports → serial; excluded under `-race`).

All pass (including a `-race` run and a `-count=3` flakiness pass), and the existing election/vote/term/stepdown suite (`TestNRG.*`) remains green. The real-election path is untouched; pre-vote is purely additive in front of it.

Hardening completed on this branch. (1) Mixed-version safety now uses a dedicated `PreVote` capability bit advertised in statsz and tracked as `nodeInfo.prevote` (not a version string). (2) A leader now explicitly refuses pre-votes (fixes a bug where the leader's own `n.leader == n.id` made it grant — which would have let a disruptor win the trial). (3) The guard window is a named constant `preVoteLeaderContactInterval` and the decision is factored into `grantPreVoteLocked` with a deterministic unit test. (4) Lost-quorum alerting is preserved on trial timeout. (5) Covered by tests: the before/after improvement, observer non-participation, and leadership-transfer bypass.

Validation runs. The full Raft suite (`go test -run TestNRG`, ~6.7 min) passes, plus a `-race` run and a `-count=3` repeat of the pre-vote tests (no flakes). Representative JetStream-cluster failover/stepdown/snapshot integration tests pass with pre-vote enabled by default (`TestJetStreamClusterStreamLeaderStepDown`, `TestJetStreamClusterMetaSnapshotsAndCatchup`, `TestJetStreamClusterLeaderStepdown`, `TestJetStreamClusterMemLeaderRestart`, `TestJetStreamClusterMultiRestartBug`). The real-election path is untouched; pre-vote is purely additive in front of it.

Remaining before production. A design decision on whether a candidate that times out on a split vote should re-enter pre-vote rather than re-campaign directly; and a dedicated capability-propagation test for the rolling-upgrade fallback (the gate logic is exercised by tests, but not the statsz round-trip itself). The real-partition soak now exists (`TestNRGPreVotePartitionSoakDoesNotDisruptMeta`) but is a single-topology proxy test rather than a randomized chaos job.

Pros, cons & trade-offs

Pros

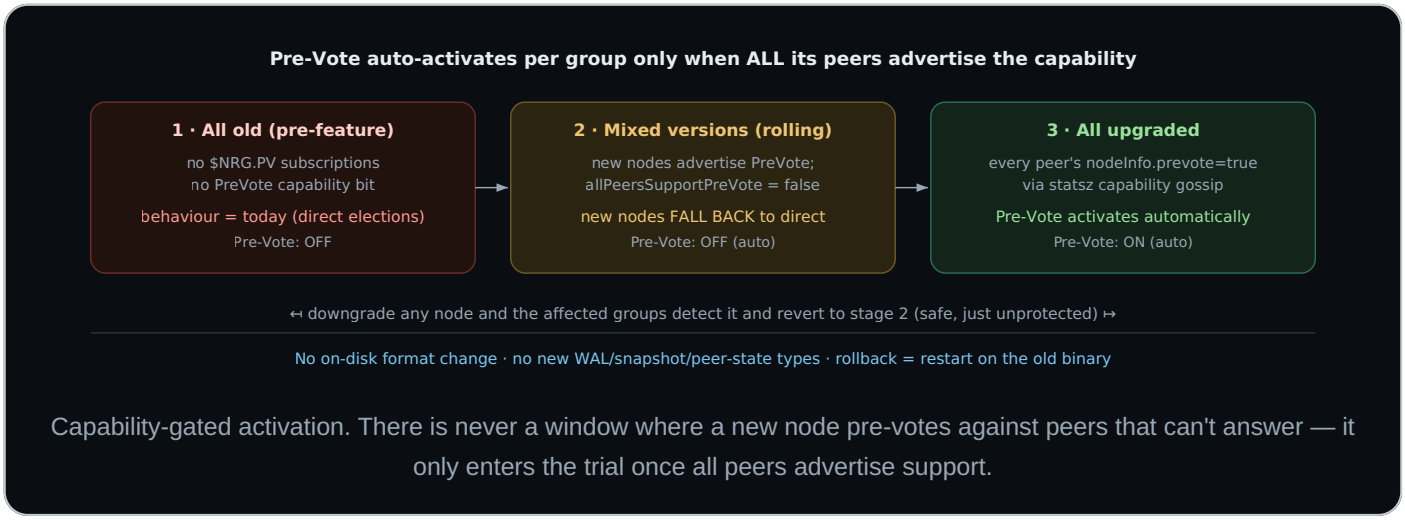
- **Stops disruptive rejoin.** A partitioned/flapping node can no longer unseat a healthy leader on reconnect — the primary fix for risk R1.
- **No term inflation.** A node that can't reach the cluster never bumps its term, so reconnection is a non-event instead of a forced re-election.
- **Protects the whole cluster.** Enabled on the meta group and every stream/consumer group, so one bad server can't trigger a cluster-wide re-election storm across thousands of groups.
- **Cheap & safe.** The trial round writes *no* durable state (no term/vote persistence), so it adds no fsyncs and cannot corrupt anything.
- **Additive & reversible.** The real-election path is untouched; the feature is gated by a flag and a capability bit and can be turned off or rolled back at any time.
- **Wire-compatible.** Reuses the existing vote message bytes on a new subject — no change to the established vote/append-entry formats.

Cons / costs

- **One extra round-trip** before a *legitimate* election when the leader is genuinely gone. Negligible against the 4–9 s election timeout, but non-zero.
- **More surface area.** A new state (`PreCandidate`), two more subscriptions per group (`$NRG.PV` + a reply inbox), new queues, and a new capability bit to maintain and test.
- **A new tunable.** The leader-contact guard (`preVoteLeaderContactInterval`, 2×hb) must stay correctly related to the heartbeat interval, and depends on `lastLeaderContact` being stamped on every accepted heartbeat.
- **Correctness interactions.** Must keep behaving for leadership transfer, observer mode, empty-log scale-up and the all-hands rule — each an extra path to get right (all covered by tests here).
- **Mixed-version complexity.** Needs capability negotiation; a detection mistake risks either a liveness stall (can't elect) or losing the protection.
- **Not a cure-all.** Does nothing for the stale-leader read window (R2) or the multi-second failover budget (R3) — those are separate.

Upgrade path considerations

The rollout is designed to be **safe under a standard rolling restart with no operator coordination**. The feature self-activates per group once every peer of that group advertises support, and self-disables (falls back to today's behavior) the moment any peer doesn't.



Capability-gated activation. There is never a window where a new node pre-votes against peers that can't answer — it only enters the trial once all peers advertise support.

Concern	How it is handled
Wire compatibility	Pre-votes ride a **new** subject (`$NRG.PV.<group>`) and reuse the existing `voteRequest`/`voteResponse` bytes. Existing vote/append-entry formats are unchanged, so an old node simply never sees a pre-vote.
Mixed-version safety	`allPeersSupportPreVote()` requires every peer to advertise the `PreVote` `ServerCapability` (via statsz → `nodeInfo.prevote`). If any peer lacks it — or is not yet known — the node falls back to a normal election, so **liveness is never lost during the rollout**.
Conservative defaults	Before statsz is exchanged, a freshly learned peer is recorded with `prevote=false` (route INFO default), so the cluster never assumes the capability prematurely.
No state migration	The feature adds **no** persisted state: no new WAL entry types, no snapshot or peer-state format change, and term/vote files are untouched. Nothing to migrate up or down.
Rollback / downgrade	Downgrading a node simply drops its `$NRG.PV` subscription; peers detect the missing capability and revert that group to direct elections. Setting `PreVote: false` disables it without a downgrade. Both are clean and immediate.
Activation order	Per-group: a stream/consumer group activates once all *its* placed peers are upgraded; the meta group activates once *all* servers are upgraded. No coordinated flag-flip is required.

> **Operational note.** Because activation is automatic and silent, surface it in logs/metrics (e.g. a debug line when a group first enters `PreCandidate`, or a per-group "pre-vote active" gauge) so operators can confirm the feature engaged after a fleet upgrade.

16 · Source map

Core engine — `server/raft.go`
state machine: `runAsFollower` 2576, `runAsCandidate` 3768, `runAsLeader` 3125 · election:

`switchToCandidate` 5270, `switchToLeader` 5304, `processVoteRequest` 5065, `requestVote` 5141 · replication: `Propose` 911, `sendAppendEntry` 4675, `processAppendEntry` 4054, `processAppendEntryResponse` 4556, `tryCommit` 3642, `applyCommit` 3524 · subjects: `createInternalSubs` 2346, constants 2299-2305 · wire: `appendEntry.encode` 2799, `voteRequest` 4820, `voteResponse` 5015 · catchup: 3306/3413/3440 · timers: 282-302 · snapshots: `encodeSnapshot` 1296, `InstallSnapshot` 1337, `installSnapshot` 1357, `CreateSnapshotCheckpoint` 1403, `NeedSnapshot` 1606, `checkpoint` 1466, `Compact` 1381 · **pre-vote (prototype):** `PreCandidate` state, `switchToPreCandidate`, `runAsPreCandidate`, `sendPreVoteRequest`, `processPreVoteRequest`, `grantPreVoteLocked`, `recentLeaderContactLocked`, `handlePreVoteRequest/Response`, `allPeersSupportPreVote`, `preVoteLeaderContactInterval`, `RaftConfig.PreVote`; capability bit `PreVote` / `SetPreVote` in `events.go` + `nodeInfo.prevote` in `server.go` / `route.go`; tests in `raft_test.go` (`TestNRGPreVote*`).

JetStream cluster — `server/jetstream_cluster.go`
`createRaftGroup` 2794 · entry ops 122-153 · **meta:** `monitorCluster` 1574, `doSnapshot` 1636, `metaSnapshot` / `encodeMetaSnapshot` 2010, `applyMetaEntries`, `checkClusterSize` 1947, `_meta_` name 457 · publish `processClusteredInboundMsg` 10048 · encode `encodeStreamMsgAllowCompressAndBatch` 9893 · `monitorStream` 3070, `applyStreamEntries` 3918, `applyStreamMsgOp` 4374 · `monitorConsumer` 6487, `applyConsumerEntries` 6778, `processReplicatedAck` 6976 · delete apply 4158 · snapshot thresholds: meta 1588-1615, stream 3119-3123, consumer 6508-6513.

Consumer — `server/consumer.go`
node attach 1524 · `processAck` 2730 · `processAckMsgLocked` 3619 (`ackInPlace` 3669) · `updateDelivered` 2964, `updateAcks` 3015 · `deliverMsg` / `replicateDeliveries` 5620/5686 · WQ enforcement 1096-1166.

Stream — `server/stream.go`
ingest `processInboundJetStreamMsg` 4840 · `processJetStreamMsg` 6124 (store 6960, PubAck 7097) · `ackMsg` 8886 (delete proposal 8963) · interest check 8746.

Transport — `server/sendq.go` (outbound), `server/events.go` (`systemSubscribe` 2845), `server/jetstream_batching.go` (`commitSingleMsg` 1059), `server/store.go` (`ConsumerState` 387).

External references

D. Ongaro & J. Ousterhout, “In Search of an Understandable Consensus Algorithm (Extended)”, 2014. · D. Ongaro, “Consensus: Bridging Theory and Practice” (PhD thesis, 2014) — §4 membership, §9.6 PreVote. · etcd-io/raft, hashicorp/raft, lni/dragonboat for field comparison.

Line numbers reflect the working tree at report-generation time and may drift with future edits. All findings were derived by reading the source in this repository; no code was modified.